Teoría de la Computación

Laboratorio 12

Problema 1: Programación funcional

Juan Jose Rivas Alvarez
Carnet universitario: 24856

4 de noviembre de 2026

1. Inciso A: Reducción beta de NOT

1.1. Definiciones y regla de sustitución

En cálculo lambda, una función se escribe $\lambda x.M$: recibe un argumento x y devuelve el cuerpo M. La reducción beta aplica una función a un argumento sustituyendo sus apariciones libres en el cuerpo:

$$(\lambda x.M)\, N \longrightarrow_\beta M[x := N].$$

La sustitución debe evitar capturar variables libres; cuando es necesario, se renombran variables ligadas (conversión alfa). En las reducciones siguientes los argumentos sustituidos son términos cerrados y no ocurre captura. La aplicación se asocia a la izquierda: $p\, a\, b = (p\, a)\, b$.

Los booleanos de Church seleccionan uno de dos argumentos:

$$\mathsf{TRUE} = \lambda x.\lambda y.x, \qquad \mathsf{FALSE} = \lambda x.\lambda y.y, \qquad \mathsf{NOT} = \lambda p.p\,\mathsf{FALSE}\,\mathsf{TRUE}.$$

1.2. Demostración de NOT TRUE

$$\begin{aligned}
\mathsf{NOT}\,\mathsf{TRUE} &= (\lambda p.p\,\mathsf{FALSE}\,\mathsf{TRUE})\,\mathsf{TRUE} \\
&\longrightarrow_\beta \mathsf{TRUE}\,\mathsf{FALSE}\,\mathsf{TRUE} && [p := \mathsf{TRUE}] \\
&= ((\lambda x.\lambda y.x)\,\mathsf{FALSE})\,\mathsf{TRUE} \\
&\longrightarrow_\beta (\lambda y.\mathsf{FALSE})\,\mathsf{TRUE} && [x := \mathsf{FALSE}] \\
&\longrightarrow_\beta \mathsf{FALSE} && [y := \mathsf{TRUE}] \\
&= \lambda x.\lambda y.y.
\end{aligned}$$

La última sustitución no cambia el cuerpo porque y no aparece libre en FALSE. Como TRUE selecciona el primer argumento y NOT coloca FALSE en esa posición, el resultado es FALSE.

1.3. Demostración de NOT FALSE

$$\begin{aligned}
\mathsf{NOT}\,\mathsf{FALSE} &= (\lambda p.p\,\mathsf{FALSE}\,\mathsf{TRUE})\,\mathsf{FALSE} \\
&\longrightarrow_\beta \mathsf{FALSE}\,\mathsf{FALSE}\,\mathsf{TRUE} && [p := \mathsf{FALSE}] \\
&= ((\lambda x.\lambda y.y)\,\mathsf{FALSE})\,\mathsf{TRUE} \\
&\longrightarrow_\beta (\lambda y.y)\,\mathsf{TRUE} && [x := \mathsf{FALSE}] \\
&\longrightarrow_\beta \mathsf{TRUE} && [y := \mathsf{TRUE}] \\
&= \lambda x.\lambda y.x.
\end{aligned}$$

Aquí x no aparece en el cuerpo $\lambda y.y$, así que la primera aplicación lo deja igual. Como FALSE selecciona el segundo argumento, devuelve TRUE.

2. Inciso B: Recursión y ciclos

2.1. Recursión y caso base

La recursión consiste en que una función resuelve un problema mediante llamadas a sí misma con entradas más pequeñas. En programación funcional permite expresar repeticiones sin actualizar un contador externo.

Un caso base resuelve directamente una entrada sencilla y detiene las llamadas. También debe existir progreso hacia él: tener un caso base no garantiza terminar si cada llamada se aleja de ese caso.

El factorial de un entero no negativo se define como:

$$n! = \begin{cases} 1 & \text{si } n = 0, \\ n\,(n-1)! & \text{si } n > 0. \end{cases}$$

```
const factorial = numero => {
    if (!Number.isInteger(numero) || numero < 0) {
        throw new Error("Se requiere un entero no negativo.");
    }
    return numero === 0 ? 1 : numero * factorial(numero - 1);
};
console.log(factorial(0)); // 1
console.log(factorial(5)); // 120
```

Por ejemplo, $3! = 3 \cdot 2! = 3 \cdot 2 \cdot 1! = 3 \cdot 2 \cdot 1 \cdot 0! = 3 \cdot 2 \cdot 1 \cdot 1 = 6$. Cada llamada reduce el argumento en uno hasta cero.

2.2. Representar un ciclo mediante recursión

Un ciclo tiene un estado, una condición de parada y una actualización. En una versión recursiva, el estado pasa como argumento, la condición se convierte en el caso base y la actualización produce la siguiente llamada. Este ejemplo suma desde 1 hasta un entero $n \geq 0$:

```
const sumarHasta = n => n === 0 ? 0 : n + sumarHasta(n - 1);
console.log(sumarHasta(4)); // 10
```

El ejemplo presupone un entero no negativo. Con un valor negativo no se alcanzaría el cero. En una función de uso general se debe validar esa entrada, como se hace en el factorial.

Estas implementaciones usan una pila de llamadas de profundidad proporcional a n. En Node.js, entradas grandes pueden desbordarla; tampoco se debe suponer optimización de llamadas finales. El factorial con `Number` puede perder precisión y finalmente producir `Infinity` para valores grandes. Los ejemplos usan valores pequeños para ilustrar el concepto.

2.3. Funciones de orden superior

Una función de orden superior recibe funciones como argumentos o devuelve una función. Los métodos `map`, `filter` y `reduce` reciben una función flecha que describe la operación para cada elemento:

- `map`: transforma cada elemento y devuelve un nuevo arreglo.
- `filter`: conserva los elementos cuyo predicado es verdadero.
- `reduce`: combina los elementos en un resultado acumulado.

```
const numeros = [1, 2, 3, 4, 5];
const dobles = numeros.map(numero => numero * 2);
const pares = numeros.filter(numero => numero % 2 === 0);
const suma = numeros.reduce((total, numero) => total + numero, 0);

console.log(dobles); // [2, 4, 6, 8, 10]
console.log(pares);  // [2, 4]
console.log(suma);   // 15
console.log(numeros); // [1, 2, 3, 4, 5]
```

Para duplicar una lista, un ciclo tradicional recorrería sus posiciones y agregaría cada doble a un arreglo de salida. `map` expresa directamente esa transformación sin administrar el índice ni modificar la lista de entrada. El valor inicial cero de `reduce` también permite sumar una lista vacía y obtener cero.

2.4. Diferencia entre los conceptos

La recursión describe cómo una función se llama a sí misma. El orden superior describe cómo una función recibe o devuelve otras funciones. No son equivalentes: `factorial` es recursiva, pero no recibe ni devuelve funciones; usar `map` no exige escribir recursión explícita. Ambos conceptos pueden combinarse y son comunes en programación funcional.

Los métodos de arreglos realizan el recorrido internamente. Utilizarlos no elimina el trabajo de recorrer los elementos ni garantiza que internamente se use recursión. Además, el estilo funcional requiere cuidar las funciones recibidas: un callback puede tener efectos secundarios o modificar objetos. En estos ejemplos los callbacks únicamente calculan valores.

2.5. Relación con la demostración ejecutable

El archivo `demostracion.js` implementa los booleanos de Church como funciones que devuelven funciones:

```
const TRUE = x => y => x;
const FALSE = x => y => y;
const NOT = p => p(FALSE)(TRUE);
const aBooleano = booleano => booleano(true)(false);
console.log(aBooleano(NOT(TRUE)));  // false
console.log(aBooleano(NOT(FALSE))); // true
```

Los booleanos de JavaScript se usan solamente para mostrar el resultado: NOT trabaja con las funciones de Church, de acuerdo con las reducciones.

3. Inciso C: Cuándo utilizar programación funcional

3.1. Situaciones recomendables y ventajas

Es recomendable cuando el problema consiste en transformar, filtrar y resumir datos: reportes, procesamiento de listas o validaciones. Las funciones puras devuelven el mismo resultado para las mismas entradas y no cambian estado externo. Esto facilita las pruebas, la reutilización y el razonamiento local. Evitar cambios en datos compartidos reduce errores por efectos secundarios.

Por ejemplo, un reporte puede sumar los importes de ventas pagadas:

```
const ventas = [
    { importe: 100, pagada: true },
    { importe: 50, pagada: false },
    { importe: 80, pagada: true }
];
const totalPagado = ventas => ventas
    .filter(venta => venta.pagada)
    .map(venta => venta.importe)
    .reduce((total, importe) => total + importe, 0);

console.log(totalPagado(ventas)); // 180
```

La operación es legible, no cambia las ventas y se puede probar con una lista vacía, ventas pendientes y ventas pagadas. La composición de pasos pequeños ayuda a expresar qué cálculo se busca. No implica ejecución paralela automática en JavaScript.

3.2. Situaciones menos convenientes y limitaciones

Un enfoque exclusivamente funcional puede resultar menos práctico al manejar entrada y salida, interfaces o simulaciones que actualizan mucho estado en cada instante. Copiar grandes estructuras continuamente puede aumentar el uso de memoria y el trabajo del recolector de basura. La recursión profunda también puede agotar la pila, y un exceso de abstracciones reduce la claridad.

Un ejemplo es una animación con miles de partículas que cambia sus posiciones en cada fotograma. Una actualización local y controlada puede ser más simple:

```
const particula = { x: 0, velocidad: 2 };
const avanzar = (estado, segundos) => {
    estado.x += estado.velocidad * segundos;
};
avanzar(particula, 0.5);
console.log(particula.x); // 1
```

`avanzar` tiene un efecto secundario: modifica el objeto recibido, por lo que no es pura. La alternativa pura sería devolver `{ ...estado, x: estado.x + estado.velocidad * segundos }`. Ambas son válidas; la elección depende de claridad, volumen de datos y mediciones de rendimiento. El estilo funcional también puede servir en juegos, pero imponerlo a cada actualización no siempre aporta ventajas.

En una aplicación real conviene mantener los cálculos puros cuando faciliten la solución y controlar los efectos secundarios donde sean necesarios.